

Supplementary Material: PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

A Technical Appendix

Quality Criterion and Break-Even Derivation

The main paper compares PCU-Select with per-PEFT LESS under an explicit quality criterion. Let Perf_{PCU} and $\text{Perf}_{\text{LESS}}$ denote their downstream scores and ϵ the acceptable difference:

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}}, \quad |\Delta_{\text{quality}}| \leq \epsilon. \quad (\text{S1})$$

For a fixed number of tasks Q , PCU-Select becomes cheaper when its shared offline cost plus its per-pair cost falls below LESS’s repeated datastore and ranking costs. Solving the cost inequality from the main paper gives

$$P > P^* = \frac{C_{\text{offline}}}{C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})}, \quad (\text{S2})$$

provided that the denominator is positive. For the four-task workload, $C_{\text{PCU}}^{\text{pair}} = 1.51/4 = 0.3775$ GPU-hours, which the tables display as 0.38. LESS costs 63.2 GPU-hours per configuration, while PCU-Select pays 144.0 GPU-hours offline plus 1.51 GPU-hours per configuration. Therefore,

$$P^* = \frac{144.0}{63.2 - 1.51} = 2.33. \quad (\text{S3})$$

PCU-Select costs less from the third integer-valued configuration onward. With 500 reduced calibration labels for each of four tasks, its marginal cost becomes $1.51 + 4(1.05) = 5.71$ GPU-hours and the worst-case threshold becomes $144.0/(63.2 - 5.71) = 2.50$ configurations.

Per-Task Results Behind the Main Table

Table 1 in the main paper summarizes four task-native metrics after scaling each to a percentage: GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1. Because these metrics have different scales, ceilings, and seed variances, their average can hide task-level differences. Table S1 therefore reports each task separately. The RDS+ and Influence columns show absolute differences in the corresponding native metric. The LESS column instead shows a

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

paired difference with a descriptive 95% interval over the five PEFT cells. We obtain this fixed-pipeline interval by resampling the evaluated cells.

The per-task results clarify the aggregate comparison. PCU-Select outperforms the PEFT-agnostic selectors on all four tasks. In contrast, its comparison with LESS depends on the task. The LESS interval excludes zero on MMLU (+0.67 Acc, [+0.24, +1.08]). It includes zero on GSM8K (+0.29 EM, [-0.04, +0.58]) and HumanEval (-0.04 Pass@1, [-0.44, +0.31]). On TyDiQA, it excludes zero in the negative direction (-0.77 F1, [-1.18, -0.43]). Averaging over tasks yields the near-zero aggregate gap in Table 1 of the main paper. PCU-Select and per-PEFT LESS are therefore near-tied on the seen registry, while PCU-Select provides amortized selection cost.

Task- and configuration-level differences. Two slices show the largest differences from LESS. On **TyDiQA**, PCU-Select trails LESS by 0.77 F1 (descriptive interval [-1.18, -0.43]). TyDiQA-GoldP emphasizes multilingual answer-span alignment, whereas the other three tasks emphasize reasoning and generation more heavily. The larger TyDiQA gap is consistent with the task sketch and shared intervention-site gradients representing its signal less directly than the per-target gradients computed by LESS. On the **MLP-only LoRA** column (L-r8-mlp), PCU-Select trails LESS by 0.57 points. Its HumanEval and TyDiQA cells have the largest dispersion, with SDs of 0.68 and 0.62. MLP-only placement concentrates all trainable capacity on the eight `mlp_out` sites. Any error in their site weights therefore affects the entire proxy, and the scorer reports its highest uncertainty on these two cells. Together, these results identify site-weighted proxy fidelity as the main source of variation across tasks and configurations.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training uses the Llama-2-7B backbone (Touvron et al. 2023), freezes the backbone, and trains only the target PEFT. Table S3 lists the per-configuration learning rates (2e-4 for LoRA, 5e-4 for (IA)³, 3e-4 for adapters, and 1e-4 for L-r64-all). The fixed recipe uses AdamW and a cosine schedule with a 0.03 warmup ratio. It runs for 1000 steps with bfloat16, gradient

Task	Metric	PCU	Δ RDS+	Δ Inf.	Δ LESS (95% CI)
GSM8K	EM	22.20	+0.55	+0.58	+0.29 [−0.04, +0.58]
HumanEval	Pass@1	17.06	+0.64	+0.24	−0.04 [−0.44, +0.31]
MMLU	Acc	49.14	+1.55	+1.00	+0.67 [+0.24, +1.08]
TyDiQA	F1	52.30	+0.22	+0.17	−0.77 [−1.18, −0.43]

Table S1: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. Δ RDS+ and Δ Inf. are absolute native-point gains over PEFT-agnostic baselines; Δ LESS is the paired difference against per-PEFT LESS with a descriptive 95% cell-resampling interval over PEFT cells. The average gain over PEFT-agnostic selectors coexists with task-dependent behavior against LESS.

clipping at 1.0, and maximum sequence length 1024. A per-device batch of 16 over 8 GPUs gives a global batch of 128.

At the 10% budget, the selected subset holds 30K examples. The 1000 steps therefore process 128K samples, or about 4.3 passes over the subset. The dataloader shuffles the full subset each epoch, so it uses every selected example. Selected instructions average ≈ 512 tokens, and length-bucketed batches reduce padding.

With FlashAttention-2 (Dao 2024) and gradient checkpointing, each step takes ≈ 5.8 wall-clock seconds. Gradient checkpointing trades compute for memory and dominates the per-step time. This corresponds to ≈ 22 samples/s in aggregate and ≈ 1.4 K tokens/s per GPU. The full run takes about 96 minutes, or 12.8 GPU-hours per PEFT-task-seed under the (wall seconds $\times$ GPU count)/3600 convention. This shared target-training cost is identical across selectors, so the selection-only comparison excludes it.

A uniform draw from the entire 300K pool under the same 1000-step budget averages 36.75 across tasks. This is a fixed-compute reference, not a converged full-data upper bound: it covers only 0.43 pool epochs, whereas the 10% subset receives about 4.3 passes.

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. We log held-out response-LM loss as an auxiliary diagnostic (`eval_loss`) but use only the primary task metric for comparison. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match)**. Chain-of-thought generation (Cobbe et al. 2021) with greedy decoding; the evaluator extracts the final numeric answer and checks exact match.
- **HumanEval (Pass@1)**. The standard unbiased Pass@1 estimator (Chen et al. 2021): sampling at temperature 0.2 with $n=20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy)**. Log-likelihood scoring (Hendrycks et al. 2021) of the four answer options; the evaluator selects the highest-likelihood option without free-form generation.
- **TyDiQA-GoldP (F1)**. Greedy decoding of the answer span (Clark et al. 2020), scored with token-level F1 against the gold answers.

These protocols follow the standard public setups for each benchmark. An evaluation adapter supplies

the task-native evaluators to the runner through the `--task-metric-factory` hook. The fixed decoding and scoring settings above prevent the run environment from changing metric definitions.

For reference, the untuned Llama-2-7B (Touvron et al. 2023) scores 13.9 on GSM8K, 12.4 on HumanEval, 43.9 on MMLU, and 46.2 on TyDiQA. Our MMLU score falls below Meta’s reported 45.3 because the prompt and harness differ. Every selector’s fine-tune, including Random, improves over the base model on all four tasks. The Random baseline therefore provides a meaningful reference for the effect of data selection.

Baseline Specifications

The three named baselines isolate two design choices: whether selection uses gradients and whether those gradients depend on the target PEFT. We specify each implementation below so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} produced by a frozen external sentence encoder. PCU-Select’s sample tower reads the same input representation but does not train it. RDS+ therefore depends on no PCU-trained features; only the selection rule differs.

RDS+ first standardizes every pool embedding per feature. It forms the query by averaging the 32 task-sketch embeddings and applies the same pool statistics. It then L_2 -normalizes the pool and query vectors and ranks candidates by cosine similarity.

RDS+ is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The “+” denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector.

Influence (shared-dataset gradient similarity). The Influence baseline builds one shared projected-gradient datastore from the base model’s intervention-site activations. It uses the JL projection and L_2 normalization specified in the reproducibility notes. Each candidate carries signatures g_x^ω at the 24 sites. The task query g_t^ω pools and renormalizes the sketch gradient at each site. Influence scores each candidate with the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ and applies global top- k without site weighting. Its PEFT-agnostic gradient source

makes the ranking independent of the target configuration. Influence therefore provides the cheap gradient baseline for measuring the value of PEFT conditioning.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplement of gradient-influence selection following LESS (Xia et al. 2024). It applies four steps to each target PEFT. First, it warms up the target adapter for 200 steps to obtain the optimization state. Second, it extracts Adam-preconditioned gradients over the adapter’s own trainable parameters at each checkpoint. Third, it projects the gradients into a target-specific low-dimensional datastore. Finally, it ranks candidates by their projected-gradient influence on the 32-example task sketch and averages across warmup checkpoints.

The first three steps build the datastore over the target adapter’s own trainable parameters. The datastore is therefore PEFT-specific but task-independent. LESS constructs it once per PEFT and reuses it for every task sketch. Only the final projected-gradient scoring step is task-specific.

It costs about 0.30 GPU-hours per task, so 1.20 covers all four task sketches for one configuration. The per-target cost of a PEFT configuration is therefore 62.0 (datastore) + 1.20 (four per-task rankings) = 63.2 GPU-hours, already spanning the full four-task evaluation; the five-PEFT registry cost is $5 \times 63.2 = 316.0$. LESS is the strongest and most expensive baseline in our study, and the amortized-cost analysis charges it exactly this per-configuration recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random has no hyperparameters. RDS+ uses the fixed representation-similarity rule above with the shared 32-example sketch as its only task input. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. We fix every baseline’s hyperparameters before test-metric evaluation.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the supplementary implementation. We take every value below from source code or its configuration files. When the code leaves a value to an external artifact or the run environment, we flag the omission instead of guessing.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora (Table S2). Deduplication runs in three stages. First, it removes exact duplicates by the content hash of each (instruction, response, source) tuple. Second, it removes normalized-string matches after lowercasing and stripping whitespace and punctuation. Third, MinHash LSH removes near-duplicates using 128 permutations and a Jaccard threshold of 0.8. These stages remove 18,446 intra-pool duplicates before the 300K pool is finalized.

For reproducibility, we include the post-filter example count and a SHA-256 checksum for each source. Examples include FLAN-v2 9f3c...a71e and OpenOrca 4b8d...c052. The supplementary manifest lists all seven sources. The full supplementary configuration comprises the PEFT registry, the scorer and feature-extraction configs, the

Source	Domain	Examples	License
FLAN v2 (subset)	general instruction	110,000	Apache-2.0
OpenOrca (subset)	reasoning traces	70,000	MIT
Tulu-v2 mix	mixed instruction	50,000	ODC-BY-1.0
OpenAssistant	dialogue	30,000	Apache-2.0
CodeAlpaca	code instruction	20,000	CC-BY-NC-4.0
Dolly-15k	general instruction	15,000	CC-BY-SA-3.0
WizardLM-evol	complex instruction	5,000	CC-BY-NC-4.0
Total		300,000	

Table S2: Candidate-pool provenance and licenses. All sources permit research use; we use the two CC-BY-NC components only for non-commercial research.

task-sketch seeds, the candidate-pool manifest with these checksums, and the evaluation adapter.

Benchmark decontamination. We decontaminate the pool against the test items of GSM8K, HumanEval, MMLU, and TyDiQA. The filter checks exact matches, normalized-string matches, 13-gram overlap, and MinHash LSH near-duplicates. The 13-gram stage removes candidates with Jaccard overlap of at least 0.5; the MinHash stage uses a threshold of 0.8. The filter removes 1,742 pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table S2 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch contains 32 examples sampled with a fixed seed (0) and stratified into response-length terciles. We draw sketches only from each task’s train or development split and keep them disjoint from the evaluation test set. HumanEval has no official train/dev split. We therefore draw its sketch from the MBPP train split and CodeAlpaca instead of HumanEval itself. Exact matching and AST-normalized function signatures remove every item that matches one of the 164 HumanEval evaluation problems. The resulting sketch source has zero overlap with the evaluation set.

The candidate pool also contains a 20K CodeAlpaca slice. We remove all 32 sketch examples from this pool by content hash and AST-normalized signature before selection. No selector can therefore recover a sketch example into its training subset. The sketch is disjoint from both the HumanEval test set and the candidate pool. Because the sketch is a selection *signal*, every selector requires this disjointness. We enforce it identically across methods.

PEFT registry. Table S3 lists all 17 PEFT configurations: five seen, nine unseen same-family configurations, and three unseen families. E5 evaluates eight held-out targets—L-r32-qkvo, AD-b16, L-r64-all, AD-b256, L-r8-highlayers, PRE-116, PT-132, and LN—while the configuration-sensitivity study uses the additional LoRA variants. All use 1000 steps, a cosine schedule, and 0.03 warmup; learning rates vary by configuration. The

LN configuration trains the `input_layernorm` and `post_attention_layernorm` scale parameters in every Transformer block and excludes the final model-level RMSNorm.

Intervention sites and gradient signatures. The site space Ω provides a shared coordinate system across PEFT families. It contains 24 sites: 8 layers $\times$ 3 module outputs per layer (`attn_out`, `mlp_out`, and `block_residual`). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) $\Phi_\omega \in \mathbb{R}^{256 \times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed `SHA256(site, global_seed=0)`; projected gradients are L_2 -normalized per row. Each sample’s signature contains 24×256 values. In bfloat16, the cached gradient-proxy store for the 300K pool occupies $300K \times 24 \times 256 \times 2B \approx 3.5$ GB. PCU-Select stores it as a memory-mapped disk array and reads it once per online selection. A PEFT configuration activates sites through its mask and a saturating capacity factor $\tanh(\eta \widehat{\text{cap}})$ with $\eta=1.0$. Operator type enters the explicit PEFT code rather than a scalar site prior; the per-family capacity mapping is in Table S8.

Conditional scorer. The conditional scorer estimates a sample’s utility for a specific task and PEFT configuration. It has three input towers: sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{128}$, and task sketch $z_t \in \mathbb{R}^{848}$. Each tower uses a LayerNorm–Linear–GELU–Linear–LayerNorm MLP. The sample and task towers project to 256 dimensions, while the PEFT tower projects to 128. FiLM modulation (Perez et al. 2018) lets the PEFT vector rescale intermediate features, while bilinear fusion models interactions between the 256-dimensional feature and the 128-dimensional PEFT representation. The fused 64-dimensional representation feeds two heads. One predicts mean utility μ ; the other predicts an example-dependent uncertainty scale σ , with a softplus floor of 10^{-3} .

We use $\hat{\sigma}$ as a ranking penalty. On held-out labels, its predicted standard deviation tracks realized error with a 10-bin expected calibration error of 0.043. Removing the penalty costs 0.28 points (Table S9), which confirms its contribution.

Training separates cheap proxy learning from expensive short-update supervision. Phase A pretrains on the gradient proxy for 3 epochs at $\text{lr } 3 \times 10^{-4}$ with loss weights (`rank`, `reg`, `proxy`, `unc`) = (0, 0, 1, 0). Phase B jointly fine-tunes for 2 epochs at $\text{lr } 10^{-4}$ with weights (1.0, 0.3, 0.5, 0.2). Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against short-update labels, a Huber proxy-distillation loss against the gradient-proxy labels, and a Gaussian negative-log-likelihood uncertainty loss. We track ranking NDCG on a held-out split of PEFT/task pairs that excludes two configurations and one task from Phase B. We keep the Phase-B checkpoint with the best held-out NDCG. The scorer uses no dropout.

Selection, clustering, and quotas. At application time, the scorer ranks each example by $q = \mu - \lambda \sigma$ with un-

certainty penalty $\lambda=0.2$. The penalty favors examples with high predicted utility and lower uncertainty. Ranking alone can concentrate the subset in a few semantic regions, so PCU-Select adds cluster-level coverage. It applies MiniBatch k -means (Sculley 2010) to the joint embeddings. It uses $k = \max(50, \lfloor \sqrt{N} \rfloor)$, batch size 4096, $n_{\text{init}}=3$, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}(B (v_k^+)^{\gamma} |C_k|^{1-\gamma}/Z)$ with $\gamma=0.6$. The cluster value v_k is the mean utility of its top 10% members. The algorithm distributes remainders by descending weight and trims over-allocation from the lowest-weight clusters.

Targets far from the seen registry can additionally calibrate the scorer with a small label set. The calibration head is a zero-initialized linear residual fit for 200 epochs at $\text{lr } 5 \times 10^{-3}$ (Adam, MSE) on the calibration labels. These deployment calibration labels $u_{\text{cal}}^{\text{hi}}$ are a reduced form of the offline short-update labels u^{hi} . They use only the warm anchor and horizon $h=1$, one of the two anchors and one of the two horizons. Each label therefore costs one quarter of the full offline routine in Eq. (S4). An offline label costs $84.0/10,000 = 0.0084$ GPU-hours, while a calibration label costs ≈ 0.0021 GPU-hours. Budgets of 200 and 500 labels therefore cost 0.42 and 1.05 GPU-hours per PEFT–task pair. The full routine would cost about four times more. We use 200 or 500 $u_{\text{cal}}^{\text{hi}}$ labels for the OOD study.

Structural support levels. The structural support levels state how safely PCU-Select can apply a PEFT code that differs from its training registry. The reported $z_p \in \mathbb{R}^{128}$ contains only the site/operator mask, capacity, and recipe vectors. Prefix, P-Tuning, and LN Tuning use the same deterministic encoding rules as the seen families; no centroid, learned family lookup, or functional fingerprint enters the reported model. E5 fixes its registry taxonomy before evaluation. L0 contains near-support same-family targets (L-r32-qkvo and AD-b16). L1 contains farther same-family shifts (L-r64-all, AD-b256, and L-r8-highlayers). L2 contains unseen families (PRE-l16, PT-l32, and LN). These levels define registry-relative structural support. LN Tuning supports reduced calibration, while Prefix/P-Tuning use a per-target selector.

Gradient-proxy and short-update labels. Short-update supervision concentrates expensive labels where they provide different kinds of information. It samples 10,000 triples using a 0.5/0.3/0.2 split. Coverage sampling spans clusters, PEFT families, capacity buckets, and tasks. Uncertainty sampling prioritizes large $\hat{\sigma}$ ($1+0.5 \text{ReLU}(\hat{u}_{\text{lo}})$), while boundary sampling targets disagreement between predicted and observed ranks. Each label measures a short-update loss reduction on the sketch from two frozen anchors— θ_{base} (the pre-trained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at $\text{lr } 10^{-4}$). Each short update Adapt_p^h uses one sampled example and 7 in-bucket fillers. The fillers stabilize the update relative to a single-example step. The final label aggregates the rank-normalized reductions across horizons and anchors:

$$u^{\text{hi}} = \frac{1}{|A|} \sum_{a \in A} \sum_{h \in \{1,4\}} w_h \tilde{\Delta}_{a,h}. \quad (\text{S4})$$

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	$r=8$	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	$r=16$	32	2e-4	seen
L-r8-mlp	LoRA	up, down	$r=8$	16	2e-4	seen
IA3-attnmlp	IA ³	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	block-residual	$b=64$	–	3e-4	seen
L-r4-qv	LoRA	q, v	$r=4$	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	$r=32$	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	$r=64$	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	$r=8$	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	$r=8$	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	$r=16$	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	$b=16$	–	3e-4	unseen-config
AD-b256	Adapter	block-residual	$b=256$	–	3e-4	unseen-config
IA3-attnonly	IA ³	k, v	–	–	5e-4	unseen-config
PRE-116	Prefix	attention	len=16	–	2e-4	ood-family
PT-132	P-Tuning	attention	len=32	–	2e-4	ood-family
LN	LN Tuning	input/post-attn RMSNorm	2 scales/layer	–	2e-4	ood-family

Table S3: Full PEFT registry (all 17 configurations). “LoRA scaling” is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^\omega$ defined in the main paper; it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA³ scales the listed activations. LN Tuning updates per-block RMSNorm scales (Wang et al. 2022; Touvron et al. 2023). Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

Level	Definition	Recommended action
L0	near-support	zero-shot selection
L1	far-support	200–500 labels (0.42–1.06 GPU-h)
L2-LN	unseen family; labels available	calibrate before use
L2-Prompt	Prefix/P-Tuning	use a per-target selector

Table S4: Deployment protocol for each registry-defined support level. Costs are per PEFT–task pair, in addition to 0.38 GPU-hours for online selection.

It uses weights $w_1=0.4$, $w_4=0.6$ and anchors $A = \{\text{base, warm}\}$. The term $\tilde{\Delta}_{a,h}$ is the reduction defined by the main paper’s short-update equation, rank-normalized within its PEFT, task, anchor, and horizon bucket.

Cost model and seeds. We measure all costs on 8×NVIDIA H20-96GB (SXM, bfloat16, NVLink) as (wall seconds × GPU count)/3600. PCU-Select pays 144.0 GPU-hours once: 48.0 for feature extraction, 4.8 for gradient-proxy labels, 84.0 for short-update labels, and 7.2 for scorer training. Dividing the 1.51 GPU-hour four-task measurement by four gives an exact average per-task cost of 0.3775 GPU-hours, displayed as 0.38. LESS costs 62.0 per PEFT datastore plus 0.30 per task ranking, hence 63.2 for four tasks. Onboarding a new four-task PEFT therefore costs 1.51 GPU-hours for PCU-Select versus 63.2 for LESS, a 42× marginal-cost reduction. The shared 12.8 GPU-hour downstream cost applies to each PEFT–task–seed run, so the selection comparison excludes it. For four tasks, break-even is $P^* = 144.0/[62.0 + 4(0.30 - 0.3775)] = 2.33$ configura-

New registry entry	PCU-Select	LESS
One new task, existing PEFT	0.38	0.30
One new PEFT, four tasks	1.51	63.2

Table S5: The amortization advantage is cross-PEFT, not cross-task. Adding a task to an existing PEFT costs each selector one online-selection pass; adding a PEFT for four tasks forces LESS to rebuild its datastore while PCU-Select reuses the offline scorer. Selection-stage GPU-hours; PCU-Select’s per-task entry rounds the exact average $1.51/4 = 0.3775$.

tions. The pipeline fixes selector-scorer training, task-sketch sampling, and pool ordering at seed 0. Reported SDs vary target-training seeds (0, 1, 2) with the selector and scorer pipeline fixed. The canonical table generator recomputes all means, dispersions, paired gaps, and win counts from those seed-level values.

Two supplementary tables bound the cost claim. Table S6 varies the offline short-update label budget. The 10K default is the smallest budget that preserves the aggregate quality tie. Smaller budgets reduce offline cost and the break-even point but also reduce quality. The offline investment is therefore a tunable trade-off. Table S7 separates the online timing components. At 300K, the scorer and clustering timers sum to 0.360 GPU-hours; the revised 0.3775 total leaves a 0.0175 residual that the available phase logs do not attribute. We report this residual separately instead of assigning it to either phase. Across pool sizes, the end-to-end cost remains approximately 170× below the scaled per-PEFT LESS datastore cost.

Labels	Offline GPU-h	Avg. quality	Gap to LESS	Break-even
2.5K	81.0	34.71	-0.43	1.31
5K	102.0	35.02	-0.12	1.65
10K	144.0	35.18	+0.04	2.33

Table S6: The offline short-update label budget sets the quality–break-even trade-off. Short-update labeling is 58% of offline cost, so a smaller budget lowers both offline GPU-hours and the break-even PEFT count, but drops below LESS-level quality. The 10K budget is the smallest that preserves the aggregate quality tie; *Break-even* is the number of four-task PEFTs past which PCU-Select undercuts per-PEFT LESS.

Pool	Feature ext. (cached)	Scorer forward	Clustering	Residual	PCU online (per p, t)
50K	9.2	0.051	0.010	0.000	0.061
100K	17.6	0.093	0.028	0.000	0.121
300K	48.0	0.249	0.111	0.018	0.378
1M	152.0	0.790	0.420	0.000	1.210

Table S7: Online selection cost versus candidate-pool size on 8×H20-96GB. Feature extraction is cached. Scorer and clustering columns report existing phase timers. Residual is total minus these timers; at 300K, the revised four-task measurement gives 0.3775 per pair, while phase timers account for 0.360, leaving 0.0175 pending attribution. Online columns show three decimals.

Algorithms

Algorithm S1 builds the reusable supervision once. Algorithm S2 applies the scorer to a new target. Only the online algorithm runs per target, and it touches no gradients. The offline algorithm amortizes the expensive gradient and label construction.

PEFT Capacity Encoding

Table S8 defines the per-family capacity used inside the site weight from the main paper. We map each family’s size knob to a trainable-parameter count at the touched sites, normalize it by d_{model} , and pass it through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Supplementary Analyses

Component ablations. Table S9 reports the full component breakdown summarized in the main analysis. Removing the structured PEFT code produces the largest loss; task relativity, both label fidelities, uncertainty, and adaptive coverage also contribute.

Cross-PEFT transfer. Table S10 supports the introduction’s 2.0-point cross-family mismatch penalty. Performance falls as the source PEFT moves farther from the target PEFT. Figure S1 shows the same mechanism from the selection side—PCU-Select’s subset overlap falls as configurations diverge, while PEFT-agnostic RDS+ remains exactly 1.000.

Algorithm S1 Offline supervision (run once)

- 1: **Input:** pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_t\}$
- 2: Extract sample signatures z_x and site gradients g_x^w ; cache them $\{O(|\mathcal{D}|)\}$ forward+backward
- 3: **for all** $(p, t) \in \mathcal{P}_{\text{seen}} \times \text{tasks}$ **do**
- 4: Compute gradient-proxy $u^{\text{lo}}(x, p, t)$ using the main-paper definition
- 5: **end for**
- 6: Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by short updates
- 7: Train s_ϕ : Stage A proxy pretrain, Stage B joint; keep best held-out-NDCG checkpoint
- 8: **Output:** scorer s_ϕ , cached features

Algorithm S2 Online selection (per target)

- 1: **Input:** target (p^*, t^*) , budget B , cached features, scorer s_ϕ
- 2: Encode z_{p^*}, z_{t^*}
- 3: **if** $\text{level}(p^*) \in \{\text{L1}, \text{L2}\}$ and short-update labels exist **then**
- 4: Fit calibration residual head from 200 or 500 reduced $u_{\text{cal}}^{\text{hi}}$ labels (single warm anchor, horizon 1)
- 5: **end if**
- 6: $(\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*}); q(x) \leftarrow \hat{\mu} - \lambda \hat{\sigma}$ for all x
- 7: Cluster the pool; allocate quotas b_k by the main-paper definition
- 8: **Output:** subset $\mathcal{S} = \bigcup_k \{\text{top-}b_k \text{ by } q \text{ in } C_k\}$

Unseen-configuration transfer. Table S11 is the paired source for Figure 3 in the main paper. Near-support targets transfer zero-shot. Calibration closes most of the gap for far same-family targets and LN Tuning.

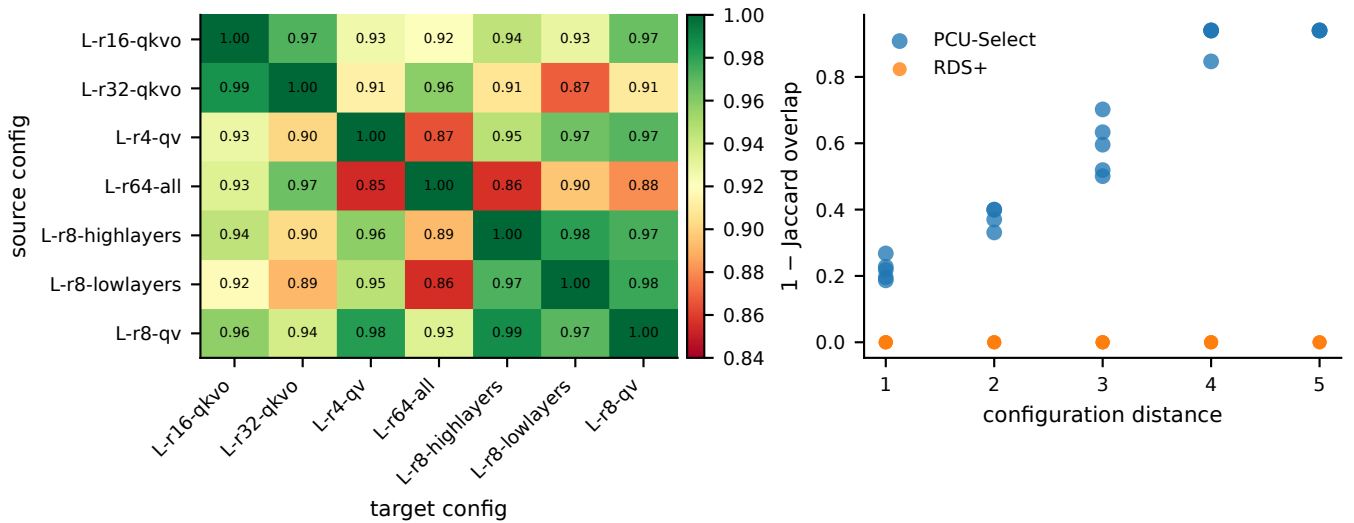


Figure S1: PCU-Select reacts to configuration changes. *Left*: target performance after training on a source-selected subset, normalized by the matched diagonal. The mean off-diagonal drop of 0.068 corresponds to a 1.45-point gap (21.25 matched vs. 19.79 mismatched). *Right*: selection difference ($1 - \text{Jaccard}$) versus a heuristic configuration distance combining log-rank, placement, and module-set changes. Across 21 unordered pairs, their descriptive Spearman correlation is $\rho = 0.969$; leave-one-configuration-out values span $[0.947, 0.972]$. RDS+ lies at zero because its same-task ranking is PEFT-independent.

Family	Size knob	$\text{cap}(p, \omega)$ at a touched site
LoRA	rank r	$r (d_{\text{in}} + d_{\text{out}})$
Adapter	bottleneck b	$2 d_{\text{model}} b$
(IA) ³	(per-vector)	#scaled activations
Prefix	length L	$L d_{\text{model}}$
LN Tuning	(per-scale)	#RMSNorm scale parameters

Table S8: Per-family capacity mapping. Capacity is the trainable-parameter count contributed at a touched site; z_p encodes operator type separately.

Variant	Metric	Drop	NDCG
Full PCU-Select	19.72±0.22	0.00	0.702
No PEFT code	18.35±0.41	1.37	0.526
Family one-hot only	19.04±0.29	0.68	0.602
No task sketch	18.90±0.31	0.82	0.597
No activation signature	19.24±0.25	0.48	0.629
Gradient-proxy labels only	18.87±0.34	0.85	0.563
Short-update labels only	19.36±0.27	0.36	0.639
No uncertainty penalty	19.44±0.24	0.28	0.690
Global top- k	18.64±0.38	1.08	0.684
Uniform clusters	19.21±0.28	0.51	0.709

Table S9: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean±SD over three seeds). Drop is relative to the full adaptive selector; we measure NDCG against held-out short-update labels.

Source ↓ / Target →	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	1.00	0.96	0.87	0.93
L-r16-qkvo	0.94	1.00	0.91	0.88
IA3-attnmlp	0.89	0.86	1.00	0.95
AD-b64	0.90	0.92	0.94	1.00

Table S10: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target- j downstream metric when trained on the subset selected for source- i , normalized by the matched diagonal. Off-diagonal cells fall below 1.00, and the two directions of a mismatched pair differ by 0.01–0.05 rather than a constant offset. Averaged over all mismatched source-target pairs the gap is 1.87 absolute points (mean matched 21.4 vs. mismatched 19.53 on the GSM8K-scale probe). Cross-family mismatches average a 2.0-point gap, the penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Tier	Targets	Ref.	Zero-shot	Cal-200	Cal-500
L0	L-r32-qkvo, AD-b16	LESS	0.34±0.41	–	–
L1	L-r64-all, AD-b256, L-r8-highlayers	LESS	2.08±0.44	0.91±0.29	0.30±0.36
L2-LN	LN	LESS	4.08±0.70	0.65±0.26	0.23±0.42
L2-Prompt	PRE-l16, PT-l32	RDS+	6.97±0.34	–	–

Table S11: Performance drop from the named reference across GSM8K, HumanEval, and MMLU (reference minus PCU-Select; paired mean±sample SD over three target-training seeds). Lower is better; zero denotes parity. L0 uses zero-shot only. Calibration reduces far same-family and LN-Tuning drops. Prefix/P-Tuning lack compatible labels.

References

- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical Society.
- Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.
- Sculley, D. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*, 1177–1178. ACM.
- Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.
- Wang, Q.; Ruan, Y.-P.; Zuo, Y.; and Li, T. 2022. Parameter-Efficient Tuning on Layer Normalization for Pre-trained Language Models. *arXiv preprint arXiv:2211.08682*.
- Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.